第 13 章：类型注解、`TypedDict` 与静态检查

先记住一句话

Python 类型注解的作用，是把“这个值应该是什么”写给人、编辑器和静态检查器看；它默认不会像 Java 编译器那样阻止代码运行，也不会替你校验 HTTP、JSON、数据库或 LLM 返回的数据。

因此，可靠的 Python 类型代码需要两道防线：

```
外部数据（不可信）
        │
        │  运行时校验：字段、实际类型、范围、状态组合
        ▼
明确的内部类型
        │
        │  Pyright：检查后续代码有没有把值用错
        ▼
业务逻辑（可信区）
```

这和 Java 项目里的“反序列化/Bean Validation + 编译器类型检查”解决的是同一组问题，只是 Python 不会自动把这两层绑在一起。

学完本章，你应该能解释：

- `name: str`、`-> list[str]` 到底约束了谁；
- `Any` 和 `object` 为什么都能接任意值，安全性却完全不同；
- “值可以是 `None`”“参数可以省略”“字典键可以缺少”为什么是三件事；
- `TypedDict` 为什么像 DTO 的结构说明，却不是运行时 DTO；
- 如何用 `Literal` 联合表达“状态不同，返回数据形状也不同”；
- 为什么外部 JSON 必须先校验，再交给带精确类型的核心逻辑。

1. Java → Python 心智模型

Java 中熟悉的概念	Python 中可以先想到	最重要的不等价之处

1. Java → Python 心智模型

Java 中熟悉的概念	Python 中可以先想到	最重要的不等价之处
方法参数、返回类型	参数注解、 -> 返回类型	Python 解释器默认不按注解拒绝调用
Object	object	取出后也要先确认具体能力，Python 常靠分支收窄
暂时关闭一段类型检查	Any	Any 不是父类，而是让检查器对这条数据流大幅放行
Map<String, Object>	dict[str, object]	不能说明每个固定字段各是什么类型
JSON DTO 的字段表	TypedDict	只描述普通 dict ，不创建 Bean，也不做运行时校验
nullable `String	`str	None`
enum 的有限值	Literal["ok", "error"]	仍是普通字符串，不会生成枚举类
函数式接口	Callable[[参数], 返回值]	只描述调用签名，不是有名字和默认方法的接口
面向接口接收集合	Mapping / Sequence	Python 的能力边界与 Java 集合并非一一对应
Java 强制转换	typing.cast	Python 的 cast 运行时不检查、不转换

可以把注解理解成“源码中的契约说明”。Pyright 像一个独立的编译期检查步骤，但 Python 解释器仍能直接执行没有通过检查的源码。

2. 一眼看懂基础注解

```
from collections.abc import Sequence


def build_labels(prefix: str, ids: Sequence[int]) -> list[str]:
    labels: list[str] = [f"{prefix}-{item_id}" for item_id in ids]
    return labels


result = build_labels("doc", (7, 13))
```

- prefix: str ： 调用方按约定应传字符串；
- ids: Sequence[int] ： 函数只要求“可按顺序读取整数”， list 和 tuple 都能传；

- `prefix: str`：调用方按约定应传字符串；
- `ids: Sequence[int]`：函数只要求“可按顺序读取整数”，`list` 和 `tuple` 都能传；
- `-> list[str]`：函数承诺正常返回字符串列表；
- `labels: list[str]`：局部变量应保存字符串列表；
- `list[str]` 中，外层是容器，方括号里是元素类型。

参数只读，所以接收较宽的 `Sequence[int]`；函数自己创建可变列表，所以返回具体 `list[str]`。思路类似 Java 参数面向 `List / Collection`，而不是锁死为 `ArrayList`。

常用容器写法：

写法	含义
`list[str]`	可变列表，每项应是 `str`
`dict[str, int]`	键应是 `str`，值应是 `int`
`tuple[int, str]`	固定两个位置，类型分别是 `int`、`str`
`tuple[str, ...]`	长度不限，每项都是 `str`
`str	None`	

`tuple[int, str]` 描述的是位置，不是“可以随便混放两种类型”。只有 `tuple[str, ...]` 中的省略号才表示同类元素可以重复任意次。

注解也不会创建或转换值：

```
count: int
print(count)  # 仍会报变量尚未绑定
```

同理，`value: int = "3"` 不会把字符串变成整数。转换必须明确写 `int("3")`，并处理可能的 `ValueError`。

3. 静态约定不等于运行时检查

```
from typing import Any


def echo_text(text: str) -> str:
```

3. 静态约定不等于运行时检查

```
from typing import Any


def echo_text(text: str) -> str:
    return text


external_value: Any = 42
result = echo_text(external_value)
print(result, type(result).__name__)  # 42 int
```

Pyright 根据函数声明，把调用结果看作 `str`。但 `Any` 让整数实参穿过了静态检查，函数体又只是原样返回，所以运行时得到的仍是 `int`。

这与 Java 有两个根本差异：

1. Java 编译器通常会拒绝把整数传给 `String` 参数；Python 可以不运行检查器而直接执行。
2. `-> str` 不会插入 `str(value)`，也不会插入 `isinstance(value, str)`。

业务要求“原本就必须是字符串”时，应显式检查：

```
def require_text(value: object) -> str:
    if not isinstance(value, str):
        raise ValueError("必须是字符串")
    return value
```

业务允许数字转字符串时，则明确调用 `str(value)`。校验和转换是两种不同的接口策略，注解不会替你选择。

4. `Any` 与 `object`：一个绕过检查，一个要求证明

问题	`Any`	`object`
能否接收任意对象	可以	可以
能否直接调用未知属性、下标或运算	检查器通常放行	必须先收窄
能否赋给更具体类型	通常可以	未收窄时不可以

合适用途	隔离确实无类型信息的兼容边界	接住未知输入，然后安全判断
主要风险	错误沿数据流扩散	多写判断，但错误更早暴露

`Any` 不是 Java 的 `Object`：

```
payload: Any = {"limit": "3"}
next_limit: int = payload["limit"] + 1
```

检查器可能放过这段代码，运行时却会因 `str + int` 抛 `TypeError`。可以把 `Any` 理解成“暂停追问这条数据流”，而不是安全的万能类型。

`object` 则迫使代码先收窄：

```
def normalize(value: object) -> str | None:
    if value is None:
        return None
    if not isinstance(value, str):
        raise ValueError("必须是字符串或 None")
    return value.strip()
```

最后一行之前，分支已经排除了 `None` 和其他类型，因此 Pyright 知道 `value` 是 `str`。这种根据控制流缩小可能类型集合的过程叫**类型收窄**。

常见收窄方式有 `is None`、`isinstance(...)`、`type(value) is int`，以及对 `Literal` 状态字段做相等判断。检查空值时优先写 `is None`；`if not value` 会把 `None`、空字符串、0 和空容器混在一起。

还有一个 Java 工程师容易意外的细节：Python 的 `bool` 是 `int` 的子类，`isinstance(True, int)` 为真。如果 JSON 字段必须是“真正的整数”，不能让 `true` 充当 1，就写 `type(value) is int`。

5. 三种“可选”必须分开

```
from typing import NotRequired, TypedDict


class SearchArguments(TypedDict):
    query: str
```

```python
from typing import NotRequired, TypedDict


class SearchArguments(TypedDict):
    query: str
    category: str | None
    limit: NotRequired[int]


def search(
    query: str,
    *,
    category: str | None = None,
    limit: int = 3,
) -> None:
    ...
```

真正的问题	写法	含义
值能不能是空值	`str	None`	
调用时能不能不传实参	`limit: int = 3`	省略后，参数绑定使用默认值 3
`TypedDict` 中能不能没有键	`limit: NotRequired[int]`	可以缺键；存在时必须是整数

所以：

- `category: str | None` 没有默认值时，调用仍必须传这个实参；
- `limit: int = 3` 可以省略，但显式传 `None` 不符合 `int`；
- `TypedDict` 中的 `category: str | None` 默认是必需键，只是值可以为空；
- `NotRequired[int]` 不会自动填默认值，也不允许字符串 `"3"`。

Java 的 `Optional<T>` 是一个真实包装对象，而 Python 的 `T | None` 是两个可能类型的联合，API 用法不能直接照搬。

6. `TypedDict`：给固定字段的字典写静态结构

DME × README × +
/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html
近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

6. `TypedDict`：给固定字段的字典写静态结构

```
from typing import NotRequired, TypedDict


class Document(TypedDict):
    document_id: str
    title: str
    score: float | None
    snippet: NotRequired[str]
```

Pyright 现在知道：

- `document_id` 、 `title` 、 `score` 必须有键；
- `score` 的值可以是 `float` 或 `None` ；
- `snippet` 可以缺键，存在时必须是 `str` 。

可以先类比成 Java DTO 的字段契约，但不能完全等同：

```
document: Document = {
    "document_id": "doc-13",
    "title": "类型注解",
    "score": None,
}
print(type(document))  # <class 'dict'>
```

运行时它仍是普通 `dict` 。它不会自动解析 JSON、检查缺字段、转换字段类型，也不能通过 `isinstance(document, Document)` 来校验。 `TypedDict` 最适合描述“外部值校验通过后，内部代码按什么结构使用”。

大多数字段都可缺少时，可以反过来写：

```
from typing import Required, TypedDict


class TraceMetadata(TypedDict, total=False):
    request_id: Required[str]
    label: str
```

```python
from typing import Required, TypedDict


class TraceMetadata(TypedDict, total=False):
    request_id: Required[str]
    label: str
    elapsed_ms: int | None
```

`total=False` 让字段默认可缺少， `Required[str]` 再把 `request_id` 改回必需键。注意 `elapsed_ms` 同时包含两个维度：键可以缺少；键存在时，值还可以是 `None` 。

如果只有少数字段可缺少，优先保持默认的 `total=True` ，只给个别字段写 `NotRequired[...]` ，更容易阅读。

7. `Literal` ：把状态和载荷绑在一起

只写 `status: str` 和 `document: Document | None` ，会允许“状态成功但文档为空”这种矛盾组合。更准确的建模方式是：

```python
from typing import Literal, TypedDict


class QuerySuccess(TypedDict):
    status: Literal["ok"]
    document: Document


class QueryNotFound(TypedDict):
    status: Literal["not_found"]
    document: None


type QueryResult = QuerySuccess | QueryNotFound
```

`Literal["ok"]` 表示只能是这个具体字符串。 `QueryResult` 表示值只能符合两个完整结构之一。状态字段因此能区分联合成员：

olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html

:期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

```python
def format_result(result: QueryResult) -> str:
    if result["status"] == "not_found":
        return "没有找到"
    return result["document"]["title"]
```

最后一行中，Pyright 已根据 `status` 排除 `QueryNotFound`，所以 `document` 一定是 `Document`。这可以类比 Java sealed hierarchy 的不同结果子类，但 Python 没有创建子类实例；值仍是普通字典，状态仍是普通字符串。外部传来的 `"timeout"` 仍需运行时拒绝。

`type QueryResult = ...` 是 Python 3.12 引入的类型别名语法。本项目要求 Python 3.14，因此可以直接使用。别名只是给复杂类型起业务名字，不是构造函数或校验器。

8. 回调和集合：只索取真正需要的能力

```python
from collections.abc import Callable, Sequence

type ResultFormatter = Callable[[QueryResult], str]


def format_all(
    result: QueryResult,
    formatters: Sequence[ResultFormatter],
) -> list[str]:
    return [formatter(result) for formatter in formatters]
```

`Callable[[QueryResult], str]` 的内层列表是参数类型，最后的 `str` 是返回类型。它可以类比一个“接收查询结果、返回字符串”的 Java 函数式接口。

不要习惯性写 `Callable[..., Any]`：省略号放弃参数形状检查，`Any` 又放弃返回值检查。复杂关键字参数和装饰器签名需要后续的 `Protocol`、`ParamSpec` 等能力，本章不展开。

集合参数也应按能力选择：

函数需要的能力	推荐参数类型	常见实参
按键读取，不修改	`Mapping[K, V]`	`dict`、其他映射
按顺序读取，不修改	`Sequence[T]`	`list`、`tuple`
需要 append 或原地排序	`list[T]`	`list`

olders/7h/w17lz72602x6dg81pjj0sfl40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html

i期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代..

函数需要的能力	推荐参数类型	常见实参
按键读取，不修改	`Mapping[K, V]`	`dict` 、其他映射
按顺序读取，不修改	`Sequence[T]`	`list` 、 `tuple`
需要 `append` 或原地排序	`list[T]`	`list`
需要新增、删除或修改键	`dict[K, V]`	`dict`

这与 Java “参数面向接口”是同一思路。但 `str` 也有序列行为；如果业务要求“多个独立字符串”，而单个字符串会导致逐字符遍历，还要在运行时单独拒绝 `str` 。

9. `cast` 与 `reveal_type`

`cast()` 不等于 Java 强制转换

```
from typing import cast

raw: object = 42
claimed = cast(str, raw)
print(type(claimed))  # <class 'int'>
```

`cast(str, raw)` 只告诉检查器“后续把它当 `str` ”，运行时原样返回对象。Java 的 `(String) value` 会检查对象并可能抛 `ClassCastException` ；Python 的 `typing.cast` 连这一步都没有。

因此，不能拿 `cast(SearchArguments, json.loads(text))` 代替外部数据校验。合理用法是：运行时分支已经证明某个事实，但检查器无法完整推导时，用局部 `cast` 补充这个事实。先有证据，再有 `cast` 。

`reveal_type()` 询问检查器的当前判断

```
from typing import reveal_type

value: object = "python"
if isinstance(value, str):
    reveal_type(value)  # Pyright 报告 str
```

README
lers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html
接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家

它适合学习和排查类型收窄。本章把这类实验放在 `static_errors.py` 中交给 Pyright 分析，不把该文件作为普通程序执行。

10. 外部 JSON：先验证，再获得精确类型

`json.loads()` 只保证 JSON 语法正确，不保证业务契约。下面都是合法 JSON，却不一定是合法搜索参数：

```
{"query": "typing", "category": null}
{"query": 13, "category": null}
{"query": "typing"}
{"query": "typing", "category": null, "limit": true}
```

它们分别是：正常值、`query` 类型错误、缺少必需键、用布尔值冒充整数。

边界代码通常按下面的顺序：

1. 用 `object` 接住未知结果；
2. 检查最外层是不是字典；
3. 检查必需键、未知键策略和字段的实际类型；
4. 检查范围、空字符串以及状态与载荷的组合；
5. 重新构造一个精确的 `TypedDict`；
6. 从这里开始，核心逻辑只接收已验证类型。

一个精简的校验器如下：

```
from collections.abc import Mapping
from typing import cast


def parse_arguments(raw: object) -> SearchArguments:
    if not isinstance(raw, dict):
        raise ValueError("arguments 必须是对象")
    if not all(isinstance(key, str) for key in raw):
        raise ValueError("arguments 的键必须是字符串")

    data = cast(Mapping[str, object], raw)
    if "query" not in data or "category" not in data:
        raise ValueError("缺少必需字段")
```

近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代

```
    query, category = data["query"], data["category"]
    if not isinstance(query, str) or not query.strip():
        raise ValueError("query 必须是非空字符串")
    if category is not None and not isinstance(category, str):
        raise ValueError("category 必须是字符串或 None")

    parsed: SearchArguments = {"query": query.strip(), "category": category}
    if "limit" in data:
        limit = data["limit"]
        if type(limit) is not int or not 1 <= limit <= 10:
            raise ValueError("limit 必须是 1 到 10 的整数")
        parsed["limit"] = limit
    return parsed
```

这里的 `cast` 只表达前两个分支已经证明的事实："这是字典，而且键都是字符串"。字段仍被逐个检查，并没有把整个外部对象直接伪装成 `SearchArguments` 。

真实服务还要明确未知字段策略、错误格式、嵌套结构、长度限制和版本兼容。也可以使用 Pydantic 等运行时模型库减少手写校验，但"运行时校验"和"静态使用检查"仍是两层职责。

11. Pyright 在开发流程中的位置

根目录 `pyproject.toml` 配置为：

```
[tool.pyright]
pythonVersion = "3.14"
typeCheckingMode = "basic"
```

`pythonVersion` 告诉 Pyright 按哪版语法和标准库类型分析，并不决定真正执行程序的解释器。`basic` 能发现本章核心契约错误，但不承诺发现所有问题。是否升级到严格模式，应由整个项目统一决定。

可以把流程类比为：

```
Java: javac 类型检查 + 程序中的边界校验
Python: Pyright 静态检查 + 程序中的边界校验 + Python 解释器执行
```

Ruff 负责格式和常规代码规则，不是类型检查器；Pyright、Ruff 和运行测试互不替代。

/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html

近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭

Ruff 负责格式和常规代码规则，不是类型检查器；Pyright、Ruff 和运行测试互不替代。

12. Java 工程师最容易踩的坑

- **把注解当运行时门禁**：`def f(x: int)` 不会自动拒绝字符串。
- **期待注解自动转换**：`"3"` 不会因为目标写了 `int` 就变成整数。
- **把 `Any` 当 `Object`**：`Any` 会让检查器放弃很多保护，应限制在很窄的边界。
- **把 `object` 当 `Any` 用**：取出后必须先证明具体类型。
- **把 `T | None` 当“可以省略”**：可空、参数默认值和 `TypedDict` 缺键互不替代。
- **认为 `TypedDict` 会校验 JSON**：运行时它仍是普通字典。
- **用 `cast` 消灭错误**：没有运行时证据的 `cast` 只是在掩盖风险。
- **回调都写 `Callable[..., Any]`**：参数数量和返回值约束都会丢失。
- **参数一律写具体 `list` / `dict`**：只读函数会无意拒绝具有同等能力的对象。
- **用 `if not value` 判断 `None`**：空字符串、0 和空容器也会被误判。

13. 运行本章示例

正常入口只使用 Python 标准库，不联网、不访问数据库、不需要 API Key，也没有额外运行依赖。
项目要求 Python 3.14 或更高版本。

若仓库根目录还没有虚拟环境，可以先创建：

```
python3.14 -m venv .venv
```

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/13_typing/example.py
```

`-B` 只是禁止生成 `.pyc` 缓存。运行时重点观察：

- 注解为 `str` 的函数为什么仍可能实际收到并返回 `int`；
- `Any` 如何把错误推迟到运行时，`object` 如何迫使代码先判断；
- `TypedDict` 的真实运行时类型为什么仍是 `dict`；
- 必需但可空的 `category` 与可缺少的 `limit` 有什么区别；
- 成功、未找到、缺字段和字段类型错误如何得到不同结果；
- `Literal` 状态如何帮助格式化逻辑收窄联合类型。

e/var/folders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html

三个文件的职责：

文件	用途
`README.md`	不看源码也能先建立类型系统心智模型
`example.py`	运行正常路径与受控失败，观察真实行为
`static_errors.py`	故意保留静态错误，只交给 Pyright，不要普通执行

静态检查

仓库当前环境提供 Pyright 时，从根目录运行：

```
.venv/bin/pyright --project pyproject.toml \
  java_to_python/chapters/13_typing/example.py
```

正常入口应得到 `0 errors` 。再检查故意写错的教学文件：

```
.venv/bin/pyright --project pyproject.toml \
  java_to_python/chapters/13_typing/static_errors.py
```

这条命令应非零退出，并报告 6 个错误和 4 条类型信息，分别用于观察：错误参数类型、未收窄 `object` 、 `TypedDict` 缺必需键、可缺少键的错误值、非法 `Literal` 状态和错误回调签名。其中 `Any` 参与的错误表达式故意不会得到诊断，这正是本章要观察的风险。

Pyright 和 Ruff 是开发检查工具，根项目没有把它们声明为可重建的运行依赖。新环境中没有 `.venv/bin/pyright` 并不影响标准库示例运行；请按项目统一方式准备开发工具，不要把当前机器恰好安装了工具误认为依赖已声明。

14. 三个小练习

1. 定义搜索参数： `query` 必需； `category` 必需但可为 `None` ； `limit` 可缺少但存在时必须是整数。分别测试缺 `category` 、 `category=None` 、缺 `limit` 、 `limit=None` 。
2. 从 `payload: object = {"limit": "3"}` 开始，写一个只接受 1～10 整数的解析器。确保 3 通过， `"3"` 、 `True` 、缺键和 11 按明确规则处理，核心计算不含 `Any` 。
3. 给 `QueryResult` 增加 `permission_denied` 状态。它必须带字符串 `reason` ，不能带文档；让 Pyright 能按状态收窄，让运行时解析器拒绝未知状态。

le/var/folders/7h/wl7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-ba73a3c3e11e486dd0155fa559c8.html

近期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求

正常入口应得到 `0 errors` 。再检查故意写错的教学文件：

```
.venv/bin/pyright --project pyproject.toml \
  java_to_python/chapters/13_typing/static_errors.py
```

这条命令应非零退出，并报告 6 个错误和 4 条类型信息，分别用于观察：错误参数类型、未收窄 `object` 、 `TypedDict` 缺必需键、可缺少键的错误值、非法 `Literal` 状态和错误回调签名。其中 `Any` 参与的错误表达式故意不会得到诊断，这正是本章要观察的风险。

Pyright 和 Ruff 是开发检查工具，根项目没有把它们声明为可重建的运行依赖。新环境中没有 `.venv/bin/pyright` 并不影响标准库示例运行；请按项目统一方式准备开发工具，不要把当前机器恰好安装了工具误认为依赖已声明。

14. 三个小练习

1. 定义搜索参数： `query` 必需； `category` 必需但可为 `None` ； `limit` 可缺少但存在时必须是整数。分别测试缺 `category` 、 `category=None` 、缺 `limit` 、 `limit=None` 。
2. 从 `payload: object = {"limit": "3"}` 开始，写一个只接受 1～10 整数的解析器。确保 3 通过， `"3"` 、 `True` 、缺键和 11 按明确规则处理，核心计算不含 `Any` 。
3. 给 `QueryResult` 增加 `permission_denied` 状态。它必须带字符串 `reason` ，不能带文档；让 Pyright 能按状态收窄，让运行时解析器拒绝未知状态。

15. 本章小结

1. 类型注解描述契约，不负责运行时验证或转换。
2. 未知外部值优先用 `object` 接住，检查后再进入核心逻辑。
3. `Any` 是逃生口，不是通用建模方案。
4. `T | None` 、参数默认值和 `NotRequired[T]` 分别处理可空、可省略实参和可缺键。
5. 固定字段字典用 `TypedDict` 描述，但外部数据仍要运行时校验。
6. 有限状态用 `Literal` ；状态决定载荷时，用可区分联合把两者绑在一起。
7. 回调写精确 `Callable` ，只读参数优先按 `Mapping` / `Sequence` 接收。
8. `cast` 只能补充已被证明的事实，不能代替校验。

如果你能解释“为什么 Pyright 通过不代表 JSON 安全”，并能画出“ `object` → 运行时校验 → `TypedDict` → 核心逻辑”这条数据流，本章核心就掌握了。

```python
"""第 13 章：类型注解、TypedDict 与静态检查。

这个入口只包含能够从头运行到尾、并通过 Pyright 的示例。故意写错类型的代码放在
static_errors.py 中，只交给类型检查器分析，不要执行它。
"""

# 阅读顺序可以分成三层：先看下面的类型契约，再看 parse_* 运行时边界，最后从 main() 的
# 五组演示观察结果。数据会经历“JSON/未知 object -> 校验后的 TypedDict -> 核心查询与格式化”。
# 本文件的所有类型注解主要服务于读者和 Pyright；除非函数体显式判断，否则 Python 解释器
# 不会仅凭注解检查、转换或拒绝运行时对象。

# json 负责把 JSON 文本解析为 Python 对象；解析只保证 JSON 语法正确，不保证业务字段正确。
import json

# Callable、Mapping、Sequence 描述对象具备的“能力”，供下面的只读参数和回调签名使用。
from collections.abc import Callable, Mapping, Sequence

# typing 中这些工具用来表达动态边界、有限字面量、TypedDict 键规则，以及静态类型断言。
from typing import Any, Literal, NotRequired, Required, TypedDict, cast

# `type 名字 = 类型表达式` 是 Python 3.12+ 的类型别名语句，不会创建运行时校验器。
# Literal 把任意 str 收紧到列出的字符串常量，适合稳定、有限的工具名和状态集合。
type ToolName = Literal["search_docs"]
type ResultStatus = Literal["ok", "not_found"]


# TypedDict 按具体键分别描述普通 dict 的静态结构，类似为字典声明一个轻量 DTO 契约；
# 它在运行时不会实例化出 Java Bean，也不会自动校验来自 JSON 的数据。
class SearchArguments(TypedDict):
    """search_docs 的输入结构。"""

    # 未包裹 NotRequired 的字段默认都是必需键；query 的键和值都必须存在且为 str。
    query: str
    # `str | None` 是联合类型：这个键必须存在，但值可为字符串或 None。
    # None 表示调用方明确选择“不按分类过滤”，不能把“可空”误读成“可缺键”。
    category: str | None
    # NotRequired 控制键是否可以不存在；若键存在，其值仍必须是 int。
```

```python
class SearchArguments(TypedDict):
    # NotRequired 控制键是否可以不存在；若键存在，其值仍必须是 int。
    # 它只描述结构，不会给字典自动填默认值；缺键时由工具实现采用自己的默认值。
    limit: NotRequired[int]


# 这个结构是外层工具调用契约：name 选择工具，arguments 保存该工具自己的参数。
class SearchToolCall(TypedDict):
    """已经通过边界校验的工具调用。"""

    name: ToolName
    arguments: SearchArguments
    # request_id 与 limit 一样可以完全缺少，但存在时必须为 str。
    request_id: NotRequired[str]


# total=False 改变本类字段的默认规则：字段默认可缺少，适合“多数键都是补充信息”的结构。
class TraceMetadata(TypedDict, total=False):
    """演示 total=False 与 Required 可以组合使用。"""

    # Required 把 request_id 从“默认可缺少”改回必需；它不是说值可以为空。
    request_id: Required[str]
    # label 可以缺键；如果存在则必须是 str。
    label: str
    # elapsed_ms 也可以缺键；如果存在，值还可以明确为 None。
    elapsed_ms: int | None


# Document 仍表示普通字典，只是静态检查器现在知道每个文档字段各自的类型和缺键规则。
class Document(TypedDict):
    document_id: str
    title: str
    # score 键必须存在，但 `float | None` 允许用 None 表达“暂时没有评分”。
    score: float | None
    # snippet 键本身可以不存在；NotRequired 不会允许已存在的值变成 None。
    snippet: NotRequired[str]
```

```python
# 下面两个 TypedDict 用 status 这个 Literal 字段表达互斥的结果形状：成功必有文档，
# 未找到时文档必须为 None。这比把两者写成彼此无关的宽联合更能表达状态与载荷的关系。
class QuerySuccess(TypedDict):
    status: Literal["ok"]
    document: Document


class QueryNotFound(TypedDict):
    status: Literal["not_found"]
    document: None


# `|` 在类型表达式中表示联合：QueryResult 的值只能符合上述两个结构之一。
type QueryResult = QuerySuccess | QueryNotFound
# Callable 的内层 `[QueryResult]` 是位置参数类型列表，最后的 str 是返回值类型。
# 因此 ResultFormatter 表示“接收一个查询结果，同步返回字符串”的可调用对象。
type ResultFormatter = Callable[[QueryResult], str]


# 自定义异常继承 ValueError，因此既能单独捕获 ContractError，也保留“值不符合约定”的语义。
class ContractError(ValueError):
    """外部数据不满足本章约定时抛出的运行时错误。"""


# `-> None` 表示函数只负责副作用，不向调用方返回业务结果；这里的副作用是打印章节标题。
def print_section(title: str) -> None:
    # f-string 会先求值花括号里的 title，再把结果嵌入字符串；开头的 \n 先输出一个空行。
    print(f"\n=== {title} ===")


# 参数写 Sequence[int]，说明这里只要求“能按顺序读取整数”，调用方可以传 tuple 或 list；
# 返回 list[str] 则明确表示函数创建并交出一个仍可追加、修改的新列表。
def build_labels(prefix: str, document_ids: Sequence[int]) -> list[str]:
    """演示参数、返回值、局部变量和容器注解。"""
    # 列表推导式会依次取出 document_ids 中的每个整数；f-string 把前缀与当前 ID 拼成字符串。
    # 输入 (7, 13) 会得到新列表 ["doc-7", "doc-13"]，并不会修改传入的序列。
```

```python
# 参数写 Sequence[int]，说明这里只要求“能按顺序读取整数”，调用方可以传 tuple 或 list；
# 返回 list[str] 则明确表示函数创建并交出一个仍可追加、修改的新列表。
def build_labels(prefix: str, document_ids: Sequence[int]) -> list[str]:
    """演示参数、返回值、局部变量和容器注解。"""
    # 列表推导式会依次取出 document_ids 中的每个整数；f-string 把前缀与当前 ID 拼成字符串。
    # 输入 (7, 13) 会得到新列表 ["doc-7", "doc-13"]，并不会修改传入的序列。
    labels: list[str] = [f"{prefix}-{document_id}" for document_id in document_ids]
    return labels


# 这个极简函数故意不写 isinstance：后面的实验用它证明注解不是 Java 编译期强制类型，
# 动态调用完全可能把 int 绑定到 text，并原样返回同一个对象。
def echo_text(text: str) -> str:
    """注解描述预期契约，但函数体不会自动检查或转换 text。"""
    return text


# object 可以接住任意对象，但静态检查器不允许直接把它当 str 使用；必须先通过分支逐步收窄。
def normalize_optional_text(value: object) -> str | None:
    """先判断 None，再用 isinstance 把 object 收窄为 str。"""
    # `is None` 判断是否正是 None 单例；不能用 `if not value`，否则空字符串也会被混为一谈。
    if value is None:
        return None
    # 前一分支已排除 None；这里再拒绝非 str，剩余路径中 Pyright 能确定 value 是 str。
    if not isinstance(value, str):
        raise ContractError("值必须是字符串或 None")
    # strip() 返回去掉首尾空白的新字符串，不会原地修改不可变的 str。
    return value.strip()


# `*` 后面的 location 是仅限关键字参数：调用时必须写 location="..."，让错误路径更醒目。
# 返回 Mapping 而非 dict，表示下游只依赖按键读取和遍历，不承诺会修改传入映射。
def expect_mapping(value: object, *, location: str) -> Mapping[str, object]:
    """把不可信 object 校验成只读的字符串键映射视图。"""
    # JSON 的 object 通常解析成 dict，但 value 的静态类型是 object，必须先在运行时确认。
    if not isinstance(value, dict):
        raise ContractError(f"{location} 必须是对象")
```

```python
# `*` 后面的 location 是仅限关键字参数：调用时必须写 location="..."，让错误路径更醒目。
# 返回 Mapping 而非 dict，表示下游只依赖按键读取和遍历，不承诺会修改传入映射。
def expect_mapping(value: object, *, location: str) -> Mapping[str, object]:
    """把不可信 object 校验成只读的字符串键映射视图。"""
    # JSON 的 object 通常解析成 dict，但 value 的静态类型是 object，必须先在运行时确认。
    if not isinstance(value, dict):
        raise ContractError(f"{location} 必须是对象")
    # 直接遍历 dict 得到键；all(...) 只有所有键都是 str 才为 True，发现首个反例就停止。
    if not all(isinstance(key, str) for key in value):
        raise ContractError(f"{location} 的键必须都是字符串")

    # cast 运行时会原样返回 value，不做验证或复制。这里能使用它，是因为上面的两个分支
    # 已证明对象是 dict 且所有键为 str；映射值暂时仍可能是任意对象，所以统一记为 object。
    return cast(Mapping[str, object], value)


# 该辅助函数只检查字段集合，不读取字段值。allowed 是允许出现的键名集合。
def reject_unknown_keys(
    mapping: Mapping[str, object],
    *,
    allowed: set[str],
    location: str,
) -> None:
    # set(mapping) 从映射的键构造集合；集合差集 `-` 留下“输入中有、允许集合中没有”的键。
    unknown: set[str] = set(mapping) - allowed
    if unknown:
        # sorted() 只为稳定错误输出的顺序，不改变 unknown；这样多个错字段也容易对照。
        raise ContractError(f"{location} 出现未知字段：{sorted(unknown)}")


# 返回 object 是刻意的：这个函数只能证明键存在，不知道每个业务字段应该是什么具体类型。
def require_key(mapping: Mapping[str, object], key: str, *, location: str) -> object:
    """显式区分“键缺少”和“键存在、值为 None”。"""
    # 先用成员判断处理缺键，不能直接依赖 mapping.get()：get() 返回 None 时无法区分两种情况。
    if key not in mapping:
        # f-string 中的 `!r` 使用 repr(key)，字符串键会带引号，错误信息的字段边界更清楚。
        raise ContractError(f"{location} 缺少字段 {key!r}")
```

```python
# 返回 object 是刻意的：这个函数只能证明键存在，不知道每个业务字段应该是什么具体类型。
def require_key(mapping: Mapping[str, object], key: str, *, location: str) -> object:
    """显式区分“键缺少”和“键存在、值为 None”。"""
    # 先用成员判断处理缺键，不能直接依赖 mapping.get()：get() 返回 None 时无法区分两种情况。
    if key not in mapping:
        # f-string 中的 `!r` 使用 repr(key)，字符串键会带引号，错误信息的字段边界更清楚。
        raise ContractError(f"{location} 缺少字段 {key!r}")
    return mapping[key]


# 这是从未知 object 到可信 str 的边界：既检查实际类型，也拒绝清理后为空的文本。
def require_non_empty_string(value: object, *, location: str) -> str:
    """检查并清理字符串；location 只是用于报错的字段路径标签。"""
    if not isinstance(value, str):
        raise ContractError(f"{location} 必须是字符串")
    # normalized 是一个新 str；value 原对象不变。空白字符串会在 strip() 后变成 ""。
    normalized: str = value.strip()
    # 空字符串在布尔上下文中为 False，所以 not normalized 能统一拒绝 "" 和纯空白文本。
    if not normalized:
        raise ContractError(f"{location} 不能为空")
    return normalized


# 该函数是 SearchArguments 的运行时可信边界：输入可以是任何 object，成功时返回重新构造的
# TypedDict；失败则立即抛 ContractError，不让未验证字段流入 search_documents()。
def parse_search_arguments(value: object) -> SearchArguments:
    """运行时校验外部工具参数，成功后才返回 TypedDict 类型。"""
    # 先验证整体形状，再执行白名单检查；这两步之后仍只知道每个值是 object。
    mapping: Mapping[str, object] = expect_mapping(value, location="arguments")
    reject_unknown_keys(
        mapping,
        allowed={"query", "category", "limit"},
        location="arguments",
    )

    # 故意拆成两步：先证明 query 键存在，再检查它的值是非空字符串。
```

```python
def parse_search_arguments(value: object) -> SearchArguments:

    # 故意拆成两步：先证明 query 键存在，再检查它的值是非空字符串。
    query_value: object = require_key(mapping, "query", location="arguments")
    query: str = require_non_empty_string(query_value, location="arguments.query")

    # category 与 query 一样是必需键，但值规则不同：None 原样保留，str 则清理首尾空白。
    category_value: object = require_key(mapping, "category", location="arguments")
    if category_value is None:
        # 局部注解明确告诉读者和检查器，这个变量之后可能走两种类型路径。
        category: str | None = None
    else:
        category = require_non_empty_string(category_value, location="arguments.category")

    # 新建字典只写入已验证并规范化的值，不把不可信 mapping 原样 cast 后继续传递。
    arguments: SearchArguments = {"query": query, "category": category}
    # NotRequired 意味着需要先判断键是否存在；不存在时不写入，默认值留给查询函数决定。
    if "limit" in mapping:
        limit_value: object = mapping["limit"]
        # bool 是 int 的子类，isinstance(True, int) 会成立；type(...) is int 才会拒绝布尔值。
        # 短路 or 会在任一条件成立时拒绝：既要类型恰好为 int，也要落在闭区间 1～10。
        if type(limit_value) is not int or not 1 <= limit_value <= 10:
            raise ContractError("arguments.limit 必须是 1 到 10 的整数")
        arguments["limit"] = limit_value
    return arguments


def parse_tool_call(value: object) -> SearchToolCall:
    """把外部 dict 边界收窄成可以交给工具实现的精确类型。"""
    # 第 1 层：检查 tool_call 最外层必须是只含已知键的字典。location 只用于拼错误路径，
    # 不会自动导航到输入中的同名字段。
    mapping: Mapping[str, object] = expect_mapping(value, location="tool_call")
    reject_unknown_keys(
        mapping,
        allowed={"name", "arguments", "request_id"},
        location="tool_call",
    )
```

```python
def parse_tool_call(value: object) -> SearchToolCall:

    # 第 2 层：取出的 name 暂时是 object；与唯一允许的 Literal 值比较后才接受。
    name: object = require_key(mapping, "name", location="tool_call")
    if name != "search_docs":
        raise ContractError("tool_call.name 必须是 'search_docs'")

    # 第 3 层：先取出 arguments（仍是 object），再交给专门函数检查内部三项字段。
    arguments_value: object = require_key(mapping, "arguments", location="tool_call")
    arguments: SearchArguments = parse_search_arguments(arguments_value)
    # 到这里 name 的值和 arguments 的结构都经过验证，可以构造内部使用的精确契约。
    parsed: SearchToolCall = {"name": "search_docs", "arguments": arguments}

    # request_id 是可缺少键；缺少时不向 parsed 填入任何内容，一旦出现则验证并规范化。
    if "request_id" in mapping:
        request_id_value: object = mapping["request_id"]
        parsed["request_id"] = require_non_empty_string(
            request_id_value, location="tool_call.request_id"
        )
    return parsed


def search_documents(arguments: SearchArguments) -> QueryResult:
    """可控的本地查询替身；不执行 HTTP、数据库或真实 Agent 调用。"""
    # 方括号访问这两个必需键；casefold() 返回适合无视大小写比较的新字符串。
    normalized_query: str = arguments["query"].casefold()
    requested_category: str | None = arguments["category"]
    # limit 键可以不存在；dict.get 的第二个参数 3 才是缺键时真正使用的运行时默认值。
    # NotRequired 本身没有向 arguments 插入 3。
    limit: int = arguments.get("limit", 3)

    # `tuple[Document, ...]` 中的省略号表示可有任意多个同类型元素，不是一个三点占位值。
    # 外层 tuple 表达样本集合在函数内不会增删；每个 Document 在运行时仍是普通 dict。
    documents: tuple[Document, ...] = (
        {
            "document_id": "doc-13",
            "title": "类型注解与工具契约",
```

```
255 def search_documents(arguments: SearchArguments) -> QueryResult:
266     documents: tuple[Document, ...] = (
267         {
268             "document_id": "doc-13",
269             "title": "类型注解与工具契约",
270             "score": 0.97,
271             "snippet": "静态约束不能替代外部输入校验。",
272         },
273         {
274             "document_id": "doc-07",
275             "title": "函数与回调",
276             "score": None,
277         },
278     )
279
280     # 这个本地样本里的文档都视为 python 分类。短路 and 会先排除“不筛选”的 None；
281     # 若调用方明确给了其他分类，立刻返回未找到联合成员，不再进入标题循环。
282     if requested_category is not None and requested_category != "python":
283         return {"status": "not_found", "document": None}
284
285     # `documents[:limit]` 创建一个最多含 limit 项的新 tuple 切片；循环只扫描这个范围。
286     for document in documents[:limit]:
287         normalized_title: str = document["title"].casefold()
288         # `in` 对字符串执行子串判断；找到第一篇匹配文档后立即返回成功联合成员。
289         if normalized_query in normalized_title:
290             return {"status": "ok", "document": document}
291     # 循环耗尽仍未 return，说明没有匹配，返回正常的 not_found 状态而不是抛异常。
292     return {"status": "not_found", "document": None}
293
294
295 # 与请求解析相同，这里把不可信结果文档逐字段校验，并重建成内部可信的 Document。
296 def parse_document(value: object) -> Document:
297     mapping: Mapping[str, object] = expect_mapping(value, location="result.document")
298     reject_unknown_keys(
299         mapping,
300         allowed={"document_id", "title", "score", "snippet"},
301         location="result.document",
```

```python
# 与请求解析相同，这里把不可信结果文档逐字段校验，并重建成内部可信的 Document。
def parse_document(value: object) -> Document:
    mapping: Mapping[str, object] = expect_mapping(value, location="result.document")
    reject_unknown_keys(
        mapping,
        allowed={"document_id", "title", "score", "snippet"},
        location="result.document",
    )

    # 嵌套调用按“先检查键存在，再检查值类型与内容”的顺序工作；任一步失败都会抛出
    # 带完整字段路径的 ContractError，不会继续构造半成品文档。
    document_id: str = require_non_empty_string(
        require_key(mapping, "document_id", location="result.document"),
        location="result.document.document_id",
    )
    title: str = require_non_empty_string(
        require_key(mapping, "title", location="result.document"),
        location="result.document.title",
    )

    # score 是必需但可空字段：先区分缺键；若不是 None，则要求实际类型恰好为 float。
    # 这里有意不接受 int，也不把字符串 "0.98" 隐式转换成浮点数。
    score_value: object = require_key(mapping, "score", location="result.document")
    if score_value is not None and type(score_value) is not float:
        raise ContractError("result.document.score 必须是浮点数或 None")

    # 先用三个必需字段建立新字典；经上面的分支，score_value 已收窄为 float | None。
    document: Document = {"document_id": document_id, "title": title, "score": score_value}
    # snippet 可以缺键，但只要出现就必须是清理后的非空字符串。
    if "snippet" in mapping:
        document["snippet"] = require_non_empty_string(
            mapping["snippet"], location="result.document.snippet"
        )
    return document


```

```python
def parse_query_result(value: object) -> QueryResult:
    """覆盖成功、未找到、字段缺失和字段类型不匹配。"""
    # 先限定结果最外层只能有 status/document 两个键，再分别要求它们存在。
    mapping: Mapping[str, object] = expect_mapping(value, location="result")
    reject_unknown_keys(mapping, allowed={"status", "document"}, location="result")

    status: object = require_key(mapping, "status", location="result")
    document_value: object = require_key(mapping, "document", location="result")

    # status 充当“判别字段”：每个字面量决定 document 必须具有哪一种形状。
    if status == "ok":
        # 成功分支把未知 document_value 交给文档解析器，验证后才装入 QuerySuccess。
        return {"status": "ok", "document": parse_document(document_value)}
    if status == "not_found":
        # 未找到是正常结果，不是异常；但协议要求它与 document=None 成对出现。
        if document_value is not None:
            raise ContractError("status 为 'not_found' 时 document 必须是 None")
        return {"status": "not_found", "document": None}
    # 未进入任一已知 Literal 分支，说明外部状态超出了当前协议。
    raise ContractError("result.status 必须是 'ok' 或 'not_found'")


def format_result(result: QueryResult) -> str:
    """Literal 字段让类型检查器按分支收窄联合类型。"""
    # 判断 status 后，Pyright 知道该分支是 QueryNotFound，document 必然是 None。
    if result["status"] == "not_found":
        return "没有找到文档"

    # 早返回已经排除未找到成员，因此这里的 result 是 QuerySuccess，document 是 Document，
    # 无需再写 `if document is not None`。缺少可选 snippet 时由 get() 提供展示文本。
    document: Document = result["document"]
    snippet: str = document.get("snippet", "（响应未提供摘要）")
    return f"{document['document_id']} | {document['title']} | {snippet}"


# 该格式化器满足同一个 ResultFormatter 签名，但只读取两个联合成员都拥有的 status 字段。
```

```python
# 该格式化器满足同一个 ResultFormatter 签名，但只读取两个联合成员都拥有的 status 字段。
def format_status(result: QueryResult) -> str:
    return f"状态={result['status']}"


# Sequence 表示这里只会按顺序读取回调，不需要调用方必须提供可变 list；tuple 同样可用。
def apply_formatters(
    result: QueryResult,
    formatters: Sequence[ResultFormatter],
) -> list[str]:
    """只读取回调序列，返回自己新建且还可能继续追加的 list。"""
    # 推导式每轮把同一个 result 传给一个 formatter，并按原顺序收集返回的 str。
    return [formatter(result) for formatter in formatters]


# 第一组实验把函数签名、局部变量和三类常见容器注解放在一起观察。
def demonstrate_basic_annotations() -> None:
    print_section("1. 基础注解、容器与箭头")
    # build_labels 接收 Sequence，所以这里可直接传不可变 tuple；返回值则是新建的 list[str]。
    labels: list[str] = build_labels("doc", (7, 13))
    # tuple[int, int] 按位置规定恰好两个整数，不等于“任意长度的整数元组”。
    coordinate: tuple[int, int] = (13, 2)
    # 字典的三个值类型不同，因此统一用最安全的共同上界 object；取值后应先收窄再专用操作。
    metadata: dict[str, object] = {
        "labels": labels,
        "coordinate": coordinate,
        "published": True,
    }

    # 这次调用确实传 str，因此实际结果符合 -> str；下一组会故意经 Any 传入 int 作对照。
    result: str = echo_text("箭头描述返回值的预期类型")
    # 这一组输出依次观察可变列表、固定位置元组、异构值字典，以及返回对象的真实运行时类型。
    print("list[str] 返回值:", labels)
    print("固定长度 tuple[int, int]:", coordinate)
    print("dict[str, object] 可容纳不同值类型:", metadata)
    print("调用表达式的实际结果/类型:", result, type(result).__name__)
```

```python
403
404 # 第二组把同一类未知输入分别交给 Any、object 和 cast，比较三者的静态与运行时后果。
405 def demonstrate_runtime_boundary() -> None:
406     print_section("2. 注解不会运行时检查；Any、object 与 cast")
407
408     # Any 让类型检查器放弃这条数据流的大部分保护，因此 int 可以穿过要求 str 的调用；
409     # echo_text 的函数体也没有自动检查或转换参数，返回的仍是原整数 42。
410     value_from_untyped_library: Any = 42
411     # 用 object 接住返回结果，避免继续相信错误的静态 str 假设，并在下面观察真实对象类型。
412     returned_value: object = echo_text(value_from_untyped_library)
413     print("把 int 经 Any 传给 -> str 的函数，实际返回:", returned_value)
414     print("实际返回类型仍是:", type(returned_value).__name__)
415
416     # 字典里实际保存的是字符串 "3"；Any 使下标结果和加法都被静态放行。
417     any_payload: Any = {"limit": "3"}
418     try:
419         # 真正执行时，str 与 int 没有定义 `+`，于是抛 TypeError 并转入 except。
420         print("Any 计算结果:", any_payload["limit"] + 1)
421     except TypeError as error:
422         # `as error` 把当前异常对象绑定到变量；这里只打印异常类名，让演示输出保持稳定。
423         print("Any 隐藏的问题在运行时暴露:", type(error).__name__)
424
425     # object 同样能引用任意对象，但检查器要求先证明能力，因而不会让错误静默扩散。
426     object_payload: object = {"limit": "3"}
427     if isinstance(object_payload, dict):
428         # 外层判断后才能调用 dict.get；取出的具体值仍未知，所以显式保留为 object。
429         limit_value: object = object_payload.get("limit")
430         # 使用精确类型判断排除 bool；只有真实 int 才能安全执行下面的 + 1。
431         if type(limit_value) is int:
432             print("object 收窄后的 limit:", limit_value + 1)
433         else:
434             # 本例实际走这里并报告 str，计算从未发生，因此没有 TypeError。
435             print("object 要求先检查，拒绝实际类型:", type(limit_value).__name__)
436
437     # cast(str, untrusted) 只把“请静态地当成 str”告诉检查器；它既不调用 str()，也不复制对象。
438     untrusted: object = 42
439     claimed_text: str = cast(str, untrusted)
```

```
405 def demonstrate_runtime_boundary() -> None:
439     claimed_text: str = cast(str, untrusted)
440     # 因而真实类型仍是 int；这说明不能用 cast 冒充外部数据的运行时校验。
441     print("cast(str, 42) 后的实际类型:", type(claimed_text).__name__)
442
443
444 # 第三组沿完整请求数据流前进，并把校验后的查询结果返回给最后一组回调实验继续使用。
445 def demonstrate_tool_call() -> QueryResult:
446     print_section("3. 校验工具请求，再交给带类型的核心逻辑")
447     # 三引号创建跨行 str；其中的 null 属于 JSON 语法，json.loads 后才会成为 Python 的 None。
448     raw_json = """
449     {
450       "name": "search_docs",
451       "arguments": {"query": "类型注解", "category": null},
452       "request_id": "req-013"
453     }
454     """
455
456     # json.loads 把文本解析成普通 Python 对象，但不会应用 SearchToolCall 业务规则。
457     # 立刻收口为 object 后，后续只能先检查再下标访问，避免 Any 绕开边界。
458     decoded: object = json.loads(raw_json)
459     # parse_tool_call 逐层检查并重建数据；返回后 call 才拥有可信的 TypedDict 静态类型。
460     call: SearchToolCall = parse_tool_call(decoded)
461     arguments: SearchArguments = call["arguments"]
462     # 核心查询只接收已校验参数，并返回 status 能区分形状的 QueryResult。
463     result: QuerySuccess | QueryNotFound = search_documents(arguments)
464
465     # get 的 "local" 是这里直接写出的备用值：request_id 缺键时才使用，不会去其他地方查找。
466     request_id: str = call.get("request_id", "local")
467     # total=False 没有让所有键都可缺少：Required 指定的 request_id 仍必须出现在字面量中。
468     trace: TraceMetadata = {"request_id": request_id}
469     # 输出依次验证：TypedDict 的运行时本质、清理后的参数、缺键与 None 的区别、Required，
470     # 最后再把查询联合交给 format_result，让它按 status 选择展示路径。
471     print("TypedDict 值的运行时类型:", type(call).__name__)
472     print("通过校验后的 arguments:", arguments)
473     print("limit 是可缺少键，当前是否存在:", "limit" in arguments)
474     print("category 是必需键，但当前值是否为 None:", arguments["category"] is None)
```

```
445  def demonstrate_tool_call() -> QueryResult:
474      print("category 是必需键，但当前值是否为 None:", arguments["category"] is None)
475      print("total=False 中被 Required 标记的键:", trace)
476      print("工具结果:", format_result(result))
477      return result
478
479
480  # 第四组把四段 JSON 文本映射到场景名，用同一循环观察成功、正常未找到和两种契约错误。
481  def demonstrate_result_validation() -> None:
482      print_section("4. 查询结果的四条边界路径")
483      # Mapping[str, str] 表示这里只需要只读"场景名 -> JSON 文本"；多行字符串中的缩进是
484      # JSON 允许的空白。成功场景特意缺 snippet，而两个反例分别破坏必需键与值类型。
485      cases: Mapping[str, str] = {
486          "成功（可选 snippet 缺少）": """
487              {"status": "ok", "document": {
488                "document_id": "doc-13", "title": "类型注解", "score": 0.98
489              }}
490          """,
491          "未找到": '{"status": "not_found", "document": null}',
492          "缺少必需字段 title": """
493              {"status": "ok", "document": {"document_id": "doc-13", "score": 0.98}}
494          """,
495          "score 类型不匹配": """
496              {"status": "ok", "document": {
497                "document_id": "doc-13", "title": "类型注解", "score": "0.98"
498              }}
499          """,
500      }
501
502      # items() 每轮产生一个二元组，再按位置解包为 label 和 text。
503      for label, text in cases.items():
504          # 每段文本先只做 JSON 语法解析；显式写成 object，迫使下一步通过业务验证器。
505          decoded: object = json.loads(text)
506          try:
507              parsed: QuerySuccess | QueryNotFound = parse_query_result(decoded)
508          except ContractError as error:
509              # 捕获本例预期的边界异常，使一个坏输入不会中断循环，后续场景仍可继续比较。
```

```
481 def demonstrate_result_validation() -> None:
503     for label, text in cases.items():
504         # 每段文本先只做 JSON 语法解析；显式写成 object，迫使下一步通过业务验证器。
505         decoded: object = json.loads(text)
506         try:
507             parsed: QuerySuccess | QueryNotFound = parse_query_result(decoded)
508         except ContractError as error:
509             # 捕获本例预期的边界异常，使一个坏输入不会中断循环，后续场景仍可继续比较。
510             print(f"{label}: 拒绝 -> {error}")
511         else:
512             # try 内没有抛 ContractError 才执行 else；此时 parsed 是可信 QueryResult。
513             print(f"{label}: 接受 -> {format_result(parsed)}")
514
515
516 # 第五组先演示精确 Callable 序列，再复用 object 收窄函数覆盖 None、str 与错误类型三条路径。
517 def demonstrate_callable_and_collections(result: QueryResult) -> None:
518     print_section("5. 精确回调与接收能力")
519     # tuple[ResultFormatter, ...] 表示任意长度、每项都满足相同签名；这里放入两个普通函数对象。
520     formatters: tuple[ResultFormatter, ...] = (format_result, format_status)
521     # apply_formatters 依次调用它们，因此得到两个字符串组成的新列表。
522     print("Sequence[Callable[[QueryResult], str]]:", apply_formatters(result, formatters))
523
524     # 前两次分别走 None 早返回和 str.strip() 路径，可直接观察联合类型的两种合法输入。
525     print("None 收窄:", normalize_optional_text(None))
526     print("str 收窄:", normalize_optional_text("  tool output  "))
527     try:
528         # int 不属于 `str | None` 的运行时契约，函数会主动抛自定义异常。
529         normalize_optional_text(13)
530     except ContractError as error:
531         print("类型不匹配由运行时校验拒绝:", error)
532
533
534 # main 只编排演示顺序：前两组建立基础，第三组产生结果，第四组独立验证边界，
535 # 第五组再消费第三组返回的 tool_result。定义 main 时函数体尚未执行，调用时才从上到下运行。
536 def main() -> None:
537     demonstrate_basic_annotations()
538     demonstrate_runtime_boundary()
```

```
517  def demonstrate_callable_and_collections(result: QueryResult) -> None:
531          print("类型不匹配由运行时校验拒绝:", error)
532
533
534  # main 只编排演示顺序：前两组建立基础，第三组产生结果，第四组独立验证边界，
535  # 第五组再消费第三组返回的 tool_result。定义 main 时函数体尚未执行，调用时才从上到下运行。
536  def main() -> None:
537      demonstrate_basic_annotations()
538      demonstrate_runtime_boundary()
539      tool_result: QuerySuccess | QueryNotFound = demonstrate_tool_call()
540      demonstrate_result_validation()
541      demonstrate_callable_and_collections(tool_result)
542
543
544  # 模块作为脚本运行时，解释器把 __name__ 设为 "__main__"，于是调用 main()；
545  # 若将本文件 import 到其他模块，这个条件为 False，只完成上面的定义，不自动打印演示内容。
546  if __name__ == "__main__":
547      main()
548
```

```python
"""只供 Pyright 分析的教学反例；不要把这个文件当作程序执行。

本文件故意保留类型错误。命令和预期诊断见 README.md。
"""

from collections.abc import Callable, Mapping
from typing import Any, Literal, NotRequired, TypedDict, cast, reveal_type


def normalize_query(query: str) -> str:
    return query.strip()


# 诊断 1：参数需要 str，不能直接传 int。
bad_query: str = normalize_query(13)


# 这一行故意没有静态诊断：Any 让错误操作穿过检查器。
payload_any: Any = {"limit": "3"}
hidden_by_any: int = payload_any["limit"] + 1
reveal_type(payload_any)  # Pyright: Any


def load_dynamic_payload() -> object:
    """模拟只承诺返回 object 的外部边界。"""
    return {"limit": "3"}


# 诊断 2：object 很安全，但不能在没有收窄时假定它可按键访问。
payload_object: object = load_dynamic_payload()
bad_access: Any = payload_object["limit"]


if isinstance(payload_object, dict) and all(isinstance(key, str) for key in payload_object):
    # 先用运行时分支证明它是字符串键映射，再用 cast 把已证明的事实告诉检查器。
    narrowed_mapping: Mapping[str, object] = cast(Mapping[str, object], payload_object)
    possible_limit: object = narrowed_mapping.get("limit")
```

```python

if isinstance(payload_object, dict) and all(isinstance(key, str) for key in payload_object):
    # 先用运行时分支证明它是字符串键映射，再用 cast 把已证明的事实告诉检查器。
    narrowed_mapping: Mapping[str, object] = cast(Mapping[str, object], payload_object)
    possible_limit: object = narrowed_mapping.get("limit")
    reveal_type(possible_limit)  # Pyright: object | None (Mapping.get 的默认结果为 None)
    if type(possible_limit) is int:
        reveal_type(possible_limit)  # Pyright: int
        checked_limit: int = possible_limit + 1


class Request(TypedDict):
    query: str
    # 必需键；只是值允许 None。
    category: str | None
    # 可缺少键；存在时必须是 int。
    limit: NotRequired[int]


# 诊断 3：category: str | None 不代表 category 键可以缺少。
missing_required_key: Request = {"query": "typing"}


# 诊断 4：NotRequired 只允许缺键，不会放宽已存在值的类型。
wrong_optional_value: Request = {
    "query": "typing",
    "category": None,
    "limit": "3",
}


type Status = Literal["ok", "not_found"]


# 诊断 5：Literal 集合中没有 "timeout"。
unsupported_status: Status = "timeout"

```

```
62
63
64  type Status = Literal["ok", "not_found"]
65
66
67  # 诊断 5：Literal 集合中没有 "timeout"。
68  unsupported_status: Status = "timeout"
69
70
71  type Formatter = Callable[[str, int], str]
72
73
74  def one_argument_formatter(text: str) -> str:
75      return text
76
77
78  # 诊断 6：回调签名少了一个 int 参数。
79  wrong_callback: Formatter = one_argument_formatter
80
81
82  # cast 只改静态视角，不验证运行时值，因此这行没有诊断。
83  claimed_text: str = cast(str, payload_object)
84  reveal_type(claimed_text)  # Pyright: str
85
```